Optimization Methods for Thermal Sciences

A panorama of principles, algorithms and code

Whady Felipe Florez

[Seminar / institution]

Outline

Setting the stage

Golden-section search

Escaping local minima: Metropolis, annealing, Deluge

Gradient-based methods: descent, Newton, dogleg

Direct search: the simplex method

Constraints: cutting plane, penalty and barrier

Wrap-up

Why optimization in the thermal sciences?

Every thermal design is a **compromise**: more insulation, more heat-exchanger area or more collector area costs money; too little wastes energy.

- How thick should the insulation on a steam line be?
- How large a heat-recovery exchanger, and at what water flow?
- At what temperature should a solar field operate?
- Where should a temperature-sensitive part go on a hot board?
- Which h, k, ε best fit my measured cooling curve?

Today's goal

A **map** of the main families of methods, one clear *picture* and one short *code* per method, all on thermal problems.

No heavy theory: what each method **does**, **when** it works and **when** it fails.

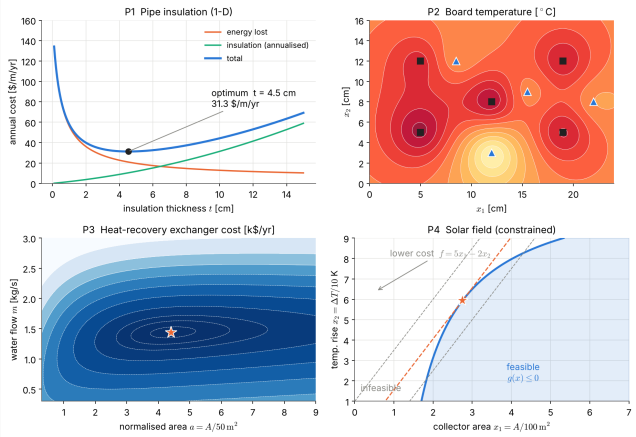
Anatomy of an optimization problem

$$\min_{x \in \mathbb{R}^n} \; f(x) \qquad \text{subject to} \qquad g_j(x) \le 0, \quad h_i(x) = 0$$

- x: the **decision variables**, a *vector* (thickness, area and flow, a position, ...)
- $f(x)$: the **objective**: a single number to minimise (annual cost, temperature, exergy loss)
- $g(x) \le 0$, $h(x) = 0$: the **constraints** (maximum temperature, demand to be met, energy balance)

Problem	x	$f(x)$	constraint
P1 insulation	$[t]$	energy lost + insulation cost	–
P2 component placement	$[x_1, x_2]$	board temperature	inside the board
P3 heat recovery	$[a,\ m]$	area + pumping − heat value	–
P4 solar field	$[A,\ \Delta T]$	area cost − premium for hot heat	demand $\le A\, G\, \eta(\Delta T)$

Four thermal test problems used throughout



P1 *1-D, unimodal*: golden section.

P2 *2-D, several valleys*: Metropolis, simulated annealing, Great Deluge.

P3 *2-D, smooth narrow valley*: gradient descent, Newton, dogleg, simplex.

P4 *2-D, constrained*: cutting plane, penalty and barrier.

Physical models are deliberately simple (closed-form), so the *method* is the star, not the model.

P1: how thick should the insulation be?

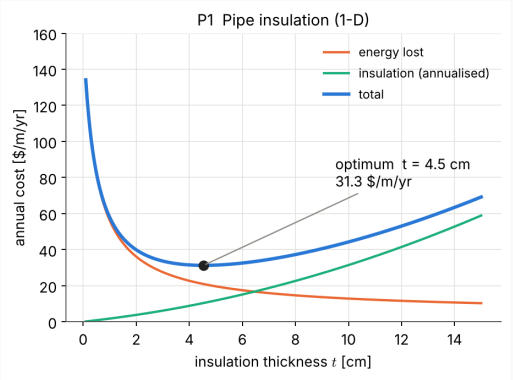
Bare steam pipe ($r_1 = 5$ cm, 180 °C) in 20 °C air. Insulation of thickness t ($k = 0.045$ W/m K):

$$q'(t) = \frac{T_f - T_\infty}{\dfrac{\ln(r_2/r_1)}{2\pi k} + \dfrac{1}{2\pi r_2 h}}, \qquad r_2 = r_1 + t$$

$$f(t) = \underbrace{c_E\, q'(t)\, H}_{\text{energy lost}} + \underbrace{\mathrm{CRF}\, c_V\, \pi (r_2^2 - r_1^2)}_{\text{insulation}}$$

$H = 8000$ h/yr, $c_E = 0.04$ \$/kWh, $c_V = 2500$ \$/m^3, CRF $= 0.2$.

- More insulation: less energy lost, more capital.
- Bare pipe: 161 \$/m/yr. Optimum: 31 \$/m/yr.



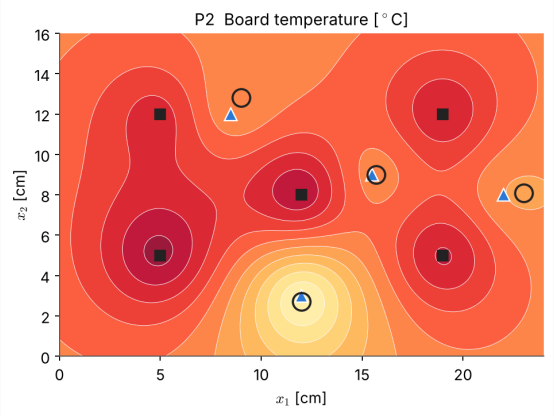
P2: where to mount a temperature-sensitive component?

Thin board ($k = 20$ W/m K, 2 mm) with 5 hot chips and 4 cooled clamps. Fin-type conduction gives, for each source q_j (W):

$$T(x) = T_0 + \sum_j \frac{q_j}{2\pi k\, t_b}\, K_0\big(m\,\|x - x_j\|\big)$$

with $m = \sqrt{2h/(k\,t_b)}$ and $q_j < 0$ for clamps.

- Goal: **minimise** $T(x)$, $x = [x_1, x_2]$ in the board.
- **Four local minima** (circled), one global: (12.0, 2.7) cm, 24.4 °C.
- The other three: 43–46 °C.



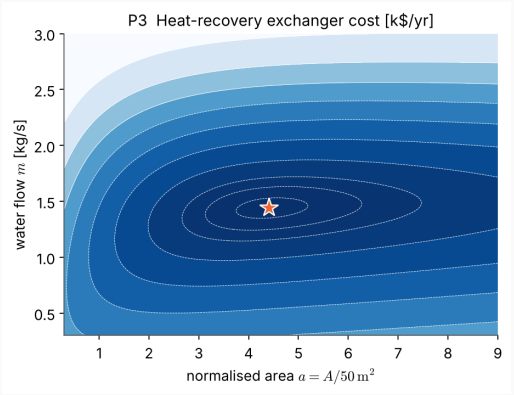
P3: heat-recovery exchanger (area and water flow)

Hot exhaust (20 kW/K, 250 °C) heats water of flow m in a counter-flow exchanger of area $A = 50\,a$ m^2 (ε–NTU):

$$\mathrm{NTU} = \frac{UA}{mc_p}, \quad U = U_0\, m^{0.8}, \quad Q = \varepsilon\, mc_p\,(T_h - T_c)$$

$$f(a, m) = \underbrace{c_A A}_{\text{area}} + \underbrace{c_p\, m^{2.8}}_{\text{pumping}} - \underbrace{c_Q\, Q}_{\text{value of heat}}$$

- Optimum: $A = 220$ m^2, $m = 1.44$ kg/s, $Q = 1.35$ MW, net gain 93.6 k\$/yr.
- A narrow curved valley: area and flow compensate each other.
- Hessian condition number ≈ 65 at the optimum.



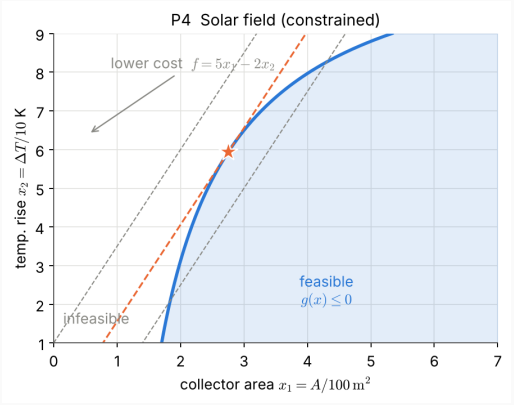
P4: solar collector field, constrained

Process heat of $Q_{dem} = 100$ kW from collectors ($G = 800$ W/m^2) with efficiency $\eta(\Delta T) = \eta_0 - a_1 \frac{\Delta T}{G} - a_2 \frac{\Delta T^2}{G}$.

$$\min\ 5\,x_1 - 2\,x_2 \quad \text{s.t.} \quad \underbrace{\frac{Q_{dem}}{G\,\eta(10x_2)} \cdot \frac{1}{100} - x_1}_{g(x)} \le 0$$

$x_1 = A/100$ m^2 (area cost 5 k\$/yr per unit),
$x_2 = \Delta T/10$ K (credit of 2 k\$/yr per unit: hotter heat is worth more).

- Optimum on the boundary: $A = 275$ m^2, $\Delta T = 59.4$ K, $\eta = 0.454$.
- g is **convex**: the feasible set is convex.



The convention for all the code: $f(x)$ with x a vector

Every method below receives **only a function of one vector**, a black box:

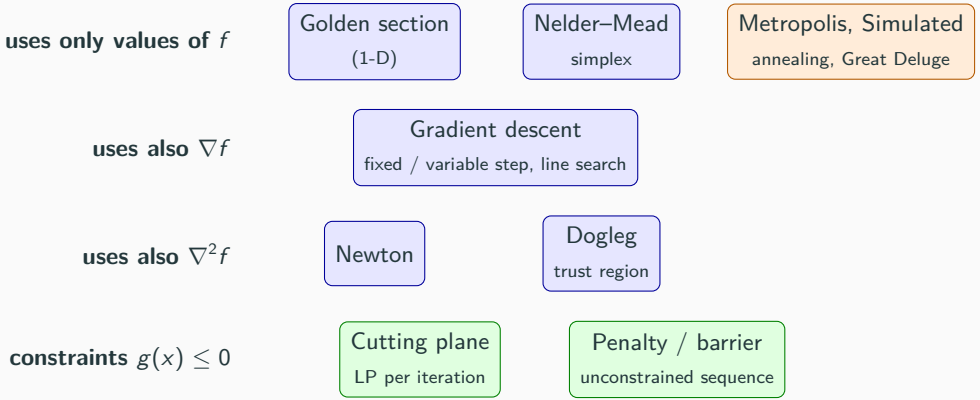
```
def cost_insulation(x):                          # P1: x = [t]
    r2 = R1 + x[0]
    R_th = np.log(r2 / R1) / (2*np.pi*K_INS) + 1.0 / (2*np.pi*r2*H_OUT)
    energy  = (T_FLUID - T_AMB) / R_th / 1000.0 * HOURS * C_ENERGY
    capital = C_VOL * np.pi * (r2**2 - R1**2) * CRF
    return energy + capital

def temperature_board(x):                        # P2: x = [x1, x2]  (cm)
    d = np.sqrt(np.sum((x - SRC[:, :2])**2, axis=1) + R0**2)       # distance to every source
    return T_BASE + np.sum(SRC[:, 2] / (2*np.pi*K_PLATE*THK) * k0(M_FIN * d))

def cost_exchanger(x):                           # P3: x = [a, m]
    A, m = 50.0 * x[0], x[1]
    return (C_AREA*A + C_PUMP*m**2.8 - C_HEAT*exchanger_duty(x)/1000.0) / 1000.0
```

Same signature for 1, 2 or 200 variables: `f(x)`. Methods never see the physics; they only call f.

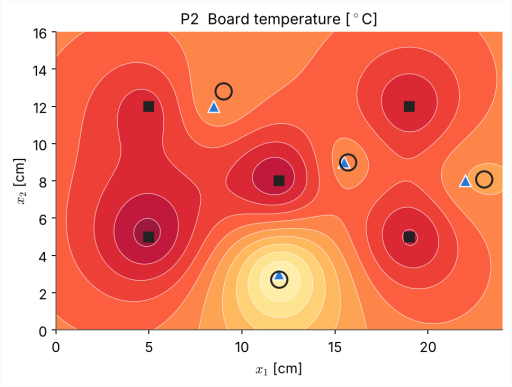
A map of the methods



Local versus global: why the landscape matters

- A **local** method stops in the valley where it starts. Start near $(23, 8)$ cm: it ends at **43 °C**, although 24 °C exists elsewhere.
- Smooth *unimodal* problems (P1, P3): local methods are enough, and the fastest ones win.
- *Multimodal* problems (P2): we need a method that sometimes accepts a **worse** point.

Rule of thumb
First plot (or sample) f. If you see several valleys, think **global**; if one smooth valley, think **Newton-like**.



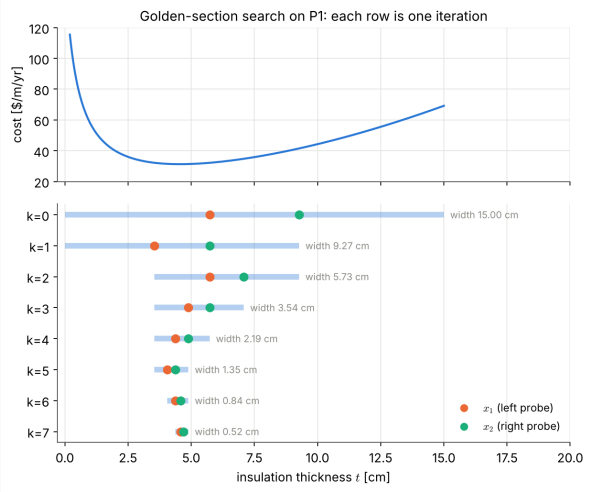
Golden section: the simplest method (1-D, no derivatives)

- Assume f is **unimodal** on $[a, b]$ (one valley). Evaluate two interior probes $x_1 < x_2$.
- If $f(x_1) < f(x_2)$ the minimum is in $[a, x_2]$; otherwise in $[x_1, b]$. **Discard the rest.**
- Choose the probes at $x_1 = b - r(b - a)$, $x_2 = a + r(b - a)$ with $r = \frac{\sqrt{5}-1}{2} = 0.618\ldots$

Why the golden ratio?
With this r one surviving probe lands **exactly** on a probe of the next iteration: only **one new evaluation** per iteration, and the bracket shrinks by 0.618 each time.

Thermal use: any one-parameter design with a single optimum (insulation thickness, fin length, optimal flow rate).

Golden section on P1: iteration by iteration



Golden section: the code

```
def golden_section(f, a, b, tol=1e-4):
    """Minimise a unimodal f over [a, b].  f takes a vector x = [t]."""
    r = (np.sqrt(5) - 1) / 2                    # 0.618...  (golden ratio)
    x1, x2 = b - r * (b - a), a + r * (b - a)   # two interior points
    f1, f2 = f(np.array([x1])), f(np.array([x2]))
    history = [(a, b)]
    while b - a > tol:
        if f1 < f2:                             # minimum lies in [a, x2]
            b, x2, f2 = x2, x1, f1              # old x1 becomes the new x2 (reused!)
            x1 = b - r * (b - a)
            f1 = f(np.array([x1]))
        else:                                   # minimum lies in [x1, b]
            a, x1, f1 = x1, x2, f2              # old x2 becomes the new x1 (reused!)
            x2 = a + r * (b - a)
            f2 = f(np.array([x2]))
        history.append((a, b))
    return 0.5 * (a + b), history
```

`f` takes a vector (`f(np.array([t]))`); `f1`, `f2` are **reused**, never recomputed.

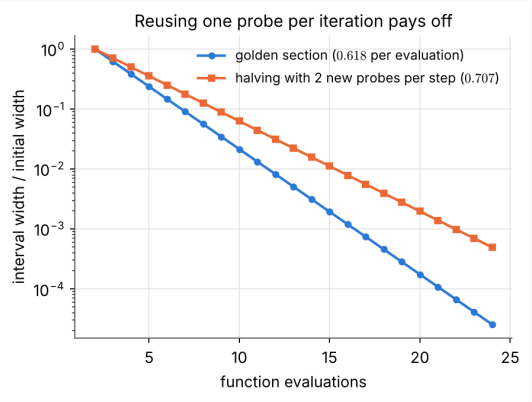
Golden section: result and cost

	manual	scipy golden
t^* [cm]	4.549	4.548
$f(t^*)$ [\$/m/yr]	31.2518	31.2518
iterations	16	20
f evaluations	18	25

Bracket $[0, 15]$ cm $\rightarrow$ 0.1 mm.

- Robust, no derivatives; linear convergence (0.618 per evaluation).
- Fails silently if f has several valleys in the bracket.

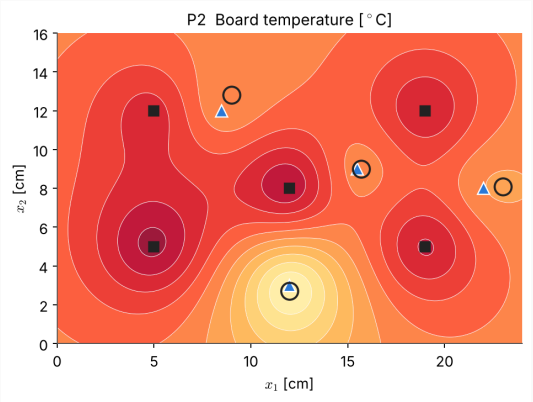
```
minimize_scalar(method="golden")
```



The problem: a valley is not the best valley

- P2: start at $x_0 = (23, 8)$ cm, in a shallow cold pocket near a clamp (43.4 °C).
- Any method that only accepts **improvements** ("greedy") stays there: every neighbour is hotter.
- To reach the cold region near (12, 2.7) we must first **climb** through ≈55 °C.

Key idea
Accept an **uphill** move sometimes, with a probability that depends on how bad it is.

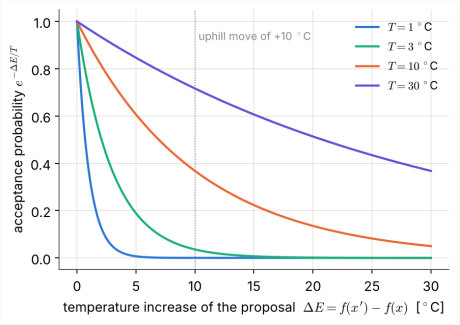


The thermodynamic idea (Metropolis, 1953)

- A system at temperature T visits a state of energy E with probability

$$p(E) \propto e^{-E/(k_B T)} \qquad \text{(Boltzmann)}$$

- **Metropolis** built a simple random walk whose visits follow exactly this law:
  - propose a random move $x \to x'$, $\Delta E = f(x') - f(x)$;
  - **accept** it if $\Delta E \leq 0$;
  - otherwise accept with probability $e^{-\Delta E/T}$.
- Here f plays the role of energy and T is a "temperature" in the units of f (°C for P2).

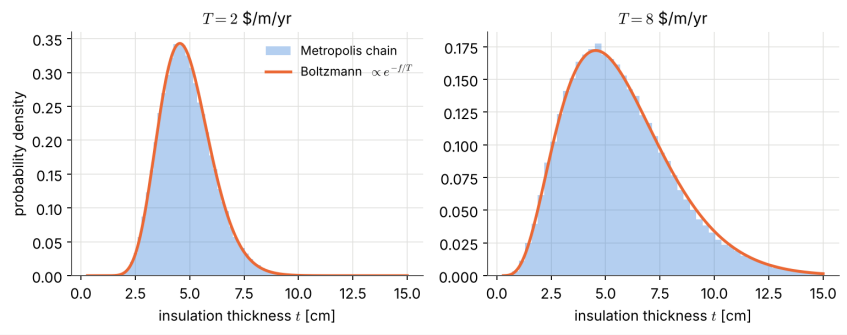


Metropolis: the code

```python
def metropolis_step(f, x, fx, T, step, rng, lo, hi):
    """One Metropolis move: propose, then accept with probability min(1, exp(-dE/T))."""
    x_new = x + step * rng.standard_normal(len(x))        # random proposal
    if np.any(x_new < lo) or np.any(x_new > hi):          # outside the board: reject
        return x, fx, False
    f_new = f(x_new)
    dE = f_new - fx
    if dE <= 0 or rng.random() < np.exp(-dE / T):         # downhill always, uphill sometimes
        return x_new, f_new, True
    return x, fx, False

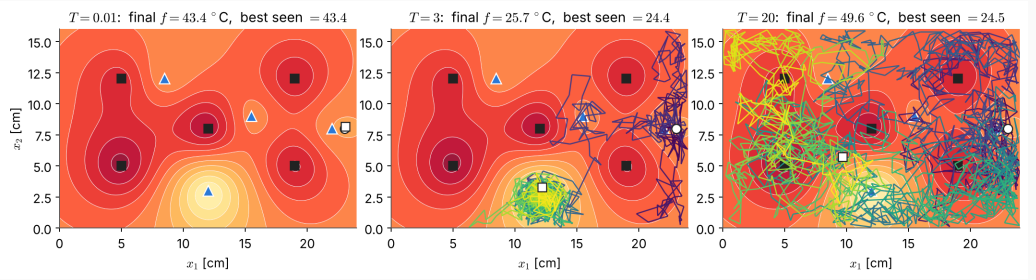

def metropolis(f, x0, T, step, n_steps, rng, lo, hi):
    """Run a Markov chain at FIXED temperature T."""
    x, fx = np.array(x0, float), f(x0)
    chain, values = [x.copy()], [fx]
    for _ in range(n_steps):
        x, fx, _ = metropolis_step(f, x, fx, T, step, rng, lo, hi)
        chain.append(x.copy())
        values.append(fx)
    return np.array(chain), np.array(values)
```

A chain at fixed T samples the Boltzmann distribution



Check on P1: histogram of 150 000 Metropolis steps (blue) against the theoretical $e^{-f/T}$ (orange). Low T: the chain hugs the minimum. High T: it wanders widely. **T is the exploration knob.**

Fixed temperature on P2: exploration versus exploitation



- $T \approx 0$ (greedy): stuck at 43.4 °C. $T = 3$: crosses to the cold region and stays. $T = 20$: accepts almost everything and never settles.
- Too cold = trapped; too hot = random walk. The right T depends on the **barrier heights** of the problem.

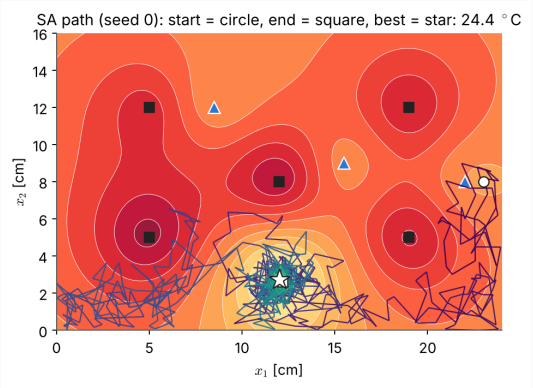
Simulated annealing: cool the temperature as you go

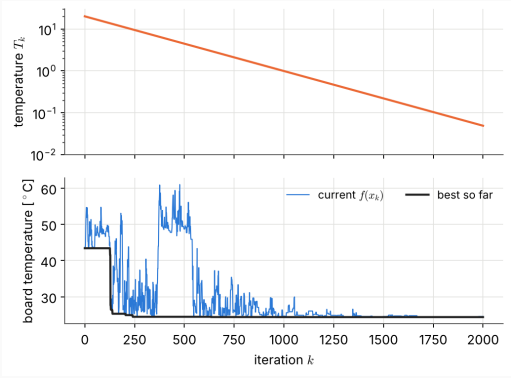
A Metropolis chain with a **decreasing** temperature, as when a metal is cooled slowly into a low-energy lattice: $T_k = T_0\,\alpha^k$, $0 < \alpha < 1$. Hot $\rightarrow$ explores; cold $\rightarrow$ refines. Keep the **best point ever seen**.

```
def simulated_annealing(f, x0, T0, alpha, step, n_steps, rng, lo, hi):
    """Metropolis chain whose temperature falls geometrically: T_k = T0 * alpha**k."""
    x, fx = np.array(x0, float), f(x0)
    best_x, best_f = x.copy(), fx
    chain, values, temps = [x.copy()], [fx], [T0]
    T = T0
    for k in range(n_steps):
        x, fx, _ = metropolis_step(f, x, fx, T, step, rng, lo, hi)
        if fx < best_f:
            best_x, best_f = x.copy(), fx      # remember the best
        T *= alpha                             # cooling schedule
        chain.append(x.copy())
        values.append(fx)
        temps.append(T)
    return best_x, best_f, np.array(chain), np.array(values), np.array(temps)
```

Parameters: $T_0 \approx$ barrier height, α such that T_{end} is small, step $\approx$ feature size.

Simulated annealing on P2



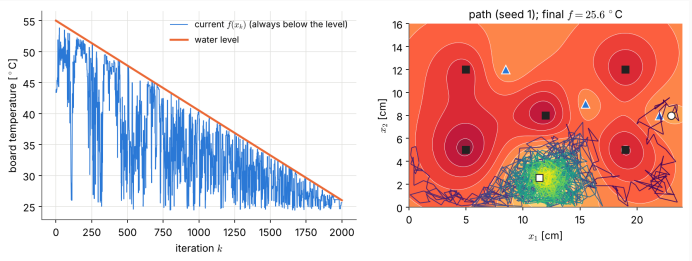


$T_0 = 20\,^\circ\mathrm{C}$, $\alpha = 0.997$, 2000 steps, step size 1 cm. Start $(23, 8)$ $\rightarrow$ best $24.4\,^\circ\mathrm{C}$ at (12.0, 2.7).

Great Deluge (Dueck, 1993): a flood instead of a temperature

- Picture a hiker in a landscape while a flood level **sinks** steadily.
- Accept **any** move whose f is below the level: no probabilities, no T.
- The level falls by a fixed "rain" each step, from a high start to a **target** value.
- The region shrinks into separate ponds; the walker ends in the one it is in.

Parameters
start level, **target**, step: the target is an *engineering goal* (here 26 °C).

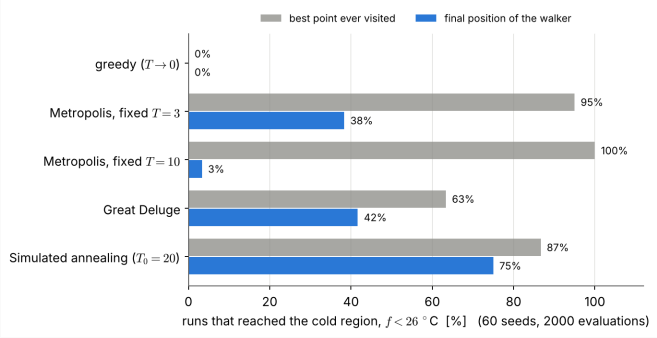


Great Deluge: the code

```
def great_deluge(f, x0, level0, target, step, n_steps, rng, lo, hi):
    """Great Deluge: accept any move below a sinking water level."""
    x, fx = np.array(x0, float), f(x0)
    level = level0                    # start high: almost all moves OK
    rain = (level0 - target) / n_steps   # level drops by 'rain' per step
    chain, values, levels = [x.copy()], [fx], [level]
    for k in range(n_steps):
        x_new = x + step * rng.standard_normal(len(x))
        if np.all(x_new >= lo) and np.all(x_new <= hi):
            f_new = f(x_new)
            if f_new <= level:                          # the only acceptance rule
                x, fx = x_new, f_new
        level -= rain
        chain.append(x.copy())
        values.append(fx)
        levels.append(level)
    return x, fx, np.array(chain), np.array(values), np.array(levels)
```

Note how short it is: one comparison `f_new <= level` replaces the whole Metropolis probability.

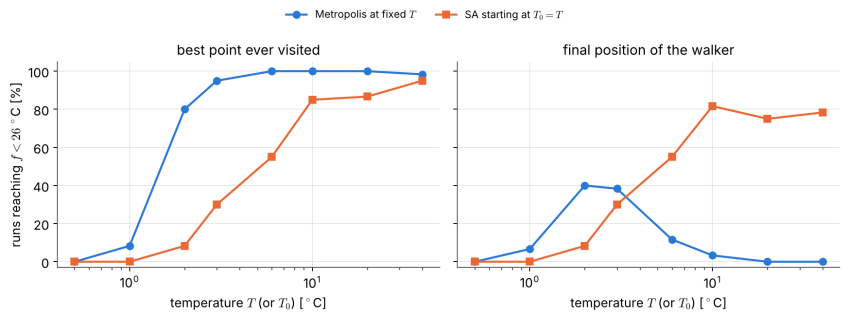
Head to head on P2 (same start, same budget)



- 60 seeds, 2000 evaluations each, start $(23, 8)$, step 1 cm; success: $f < 26\,^\circ$C.
- Greedy: **never** escapes.
- If we **remember the best point**, any T high enough to cross the barriers finds the cold region (95–100%).
- But **where the walker ends** is another story: only SA *ends* cold (75%).
- Great Deluge: 63% / 42%.

Judge stochastic methods over many seeds, never one run.

What does the temperature knob really do?



- **Best visited:** fixed $T \geq 3$ already finds the cold region; SA needs a hot enough start ($T_0 \gtrsim 10$), because it cools while exploring. Too cold ($T \leq 1$): trapped.
- **Final position:** a fixed-T chain ends wherever thermal noise puts it (best $\approx 40\%$ at $T = 2$–3, nearly 0 when hotter). SA's final state is good ($\approx 80\%$) *without* bookkeeping, because it cools into the minimum.

In `scipy` and in practice

```
from scipy.optimize import dual_annealing
res = dual_annealing(temperature_board, bounds=[(0, 24), (0, 16)], seed=0, maxfun=2000)
print(res.x, res.fun)          # -> [12.04  2.70]   24.38 C
```

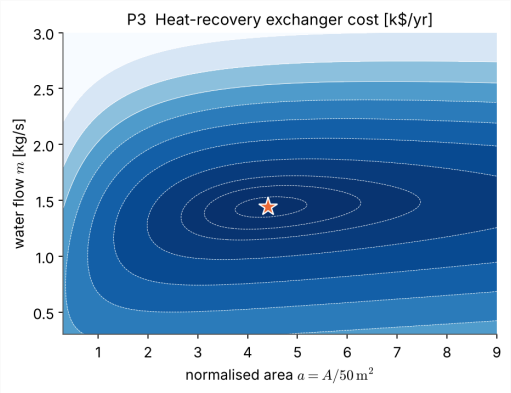
- `dual_annealing` (generalised SA plus a local polish) reaches the same optimum as our manual code.
- Practical advice: choose **step** from the feature size of f, T_0 from a typical **barrier**, T_{end} from the resolution you need; run **many seeds**; keep the best point; finish with a local method.
- Cost: thousands of evaluations; use them when f is cheap, derivatives unavailable, or valleys abound.

Using derivatives: back to the smooth valley (P3)

- $\nabla f(x)$ points **uphill**; $-\nabla f$ is the direction of fastest local decrease.
- General descent iteration:

$$x_{k+1} = x_k + \alpha_k \, p_k, \qquad p_k = -\nabla f(x_k)$$

- The art is in the **step length** α_k (and, later, in a better direction p_k).
- Derivatives are not magic: from a black-box $f(x)$ we get them by **finite differences**.



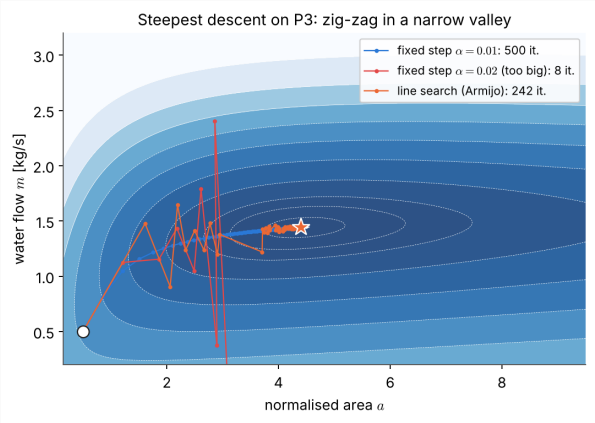
Derivatives of a black box: finite differences

```python
def numgrad(f, x, h=1e-6):
    """Central-difference gradient of a scalar function f(x), x a vector."""
    g = np.zeros(len(x))
    for i in range(len(x)):
        e = np.zeros(len(x))
        e[i] = h * (1 + abs(x[i]))
        g[i] = (f(x + e) - f(x - e)) / (2 * e[i])
    return g


def numhess(f, x, h=1e-4):
    """Hessian by differencing the gradient (symmetrised)."""
    n = len(x)
    H = np.zeros((n, n))
    for i in range(n):
        e = np.zeros(n)
        e[i] = h * (1 + abs(x[i]))
        H[:, i] = (numgrad(f, x + e) - numgrad(f, x - e)) / (2 * e[i])
    return 0.5 * (H + H.T)
```

Central difference $\frac{f(x+he_i)-f(x-he_i)}{2h}$: $2n$ evaluations. The Hessian differences the gradient.

Fixed step: the step length decides everything



- $\alpha = 0.01$: converges, but **500 iterations** are not enough (error still 10^{-5}).
- $\alpha = 0.02$: **diverges** in 8 iterations (leaves the valid region).
- Stability limit: $\alpha < 2/\lambda_{\max}(H)$. Here $\lambda_{\max} \approx 145$, so $\alpha < 0.0138$.
- Narrow valley (condition number ≈ 65): steepest descent **zig-zags**.

Lesson
A step that is good now may be terrible later. Use a **variable** step.

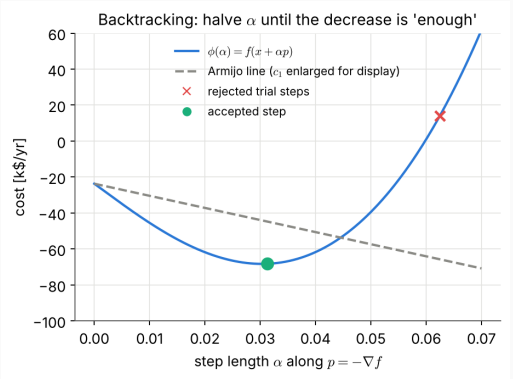
Variable step: prescribed decay or line search

Prescribed decay
$\alpha_k = \dfrac{\alpha_0}{1 + 0.05\,k}$: cheap, but you must guess α_0 and the rate (stalls in the example: error 0.06 after 500 its.).

Line search (backtracking Armijo)
Start with a large α and **halve it** until the decrease is *sufficient*:

$$f(x + \alpha p) \le f(x) + c_1\,\alpha\,\nabla f^{\top} p, \quad c_1 = 10^{-4}$$

Adapts to the local scale automatically; costs a few extra evaluations per iteration.



Trial steps $\alpha = 1,\ \frac{1}{2},\ \frac{1}{4},\ \frac{1}{8},\ \frac{1}{16}$ are rejected; $\alpha = \frac{1}{32}$ is accepted.

Backtracking line search: the code

```
def backtracking(f, x, fx, g, p, alpha=1.0, c1=1e-4, shrink=0.5):
    """Armijo backtracking: halve alpha until f drops enough."""
    while f(x + alpha * p) > fx + c1 * alpha * (g @ p):
        alpha *= shrink
    return alpha
```

Five lines: the whole of the Armijo rule. `g @ p` is the slope $\nabla f^\top p$ along the step.

Gradient descent with the three step rules: the code

```
def gradient_descent(f, x0, rule="line_search", alpha0=0.01, n_iter=200, tol=1e-6):
    """Steepest descent with three step-size rules: 'fixed', 'decay', 'line_search'."""
    x = np.array(x0, float)
    path = [x.copy()]
    for k in range(n_iter):
        g = numgrad(f, x)
        if np.linalg.norm(g) < tol:
            break
        p = -g                                          # steepest-descent direction
        if rule == "fixed":
            alpha = alpha0
        elif rule == "decay":
            alpha = alpha0 / (1 + 0.05 * k)             # prescribed, shrinking step
        else:
            alpha = backtracking(f, x, f(x), g, p)      # step chosen by line search
        x = x + alpha * p
        path.append(x.copy())
        if not np.isfinite(f(x)):                       # left the valid region: stop
            break
    return x, np.array(path)
```

`rule` changes only the line computing `alpha`. P3 from $(0.5, 0.5)$: line search needs **242 iterations**.

Newton's method: use the curvature

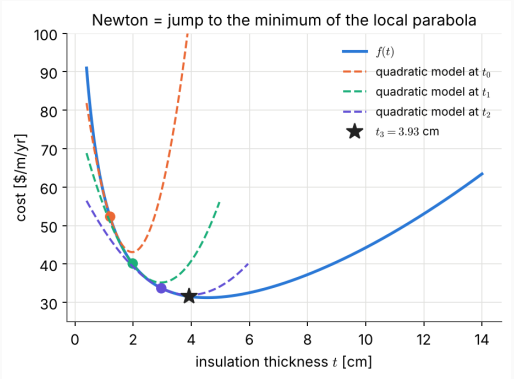
- Replace f by its **local parabola** (second-order Taylor):

$$m(p) = f + \nabla f^\top p + \tfrac{1}{2}\, p^\top H\, p$$

- Jump to the minimum of the parabola: solve

$$H\, p = -\nabla f, \qquad x_{k+1} = x_k + p$$

- Near the optimum the error is **squared** at each step (quadratic convergence).
- Cost per iteration: Hessian (n^2 derivatives) and one linear solve.



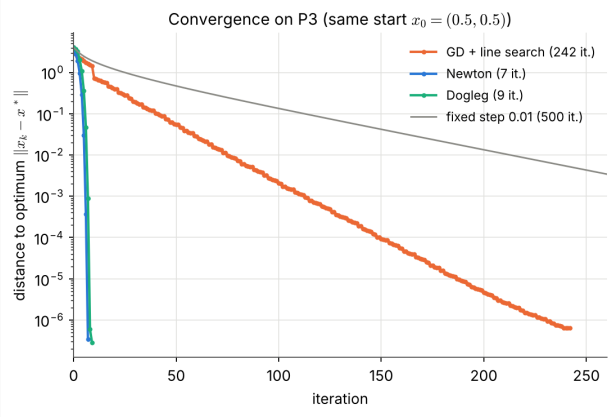
P1 from $t_0 = 1.2$ cm: three Newton steps reach 3.93 cm (the optimum is 4.55).

Newton: the code

```
def newton(f, x0, n_iter=50, tol=1e-6, safeguard=True):
    """Newton. safeguard=True: shift H if not PD, plus line search."""
    x = np.array(x0, float)
    path = [x.copy()]
    for k in range(n_iter):
        g, H = numgrad(f, x), numhess(f, x)
        if np.linalg.norm(g) < tol:
            break
        if safeguard:
            lam_min = np.linalg.eigvalsh(H)[0]
            if lam_min < 1e-6:                              # not positive definite
                H = H + (1e-6 - lam_min) * np.eye(len(x))   # shift the spectrum
        p = -np.linalg.solve(H, g)                          # solve  H p = -g
        alpha = backtracking(f, x, f(x), g, p) if safeguard else 1.0
        x = x + alpha * p
        path.append(x.copy())
        if not np.isfinite(f(x)):                           # left the valid region: stop
            break
    return x, np.array(path)
```

`safeguard=True`: shift a non-PD H and backtrack. `False`: the textbook method.

Newton versus gradient descent on P3

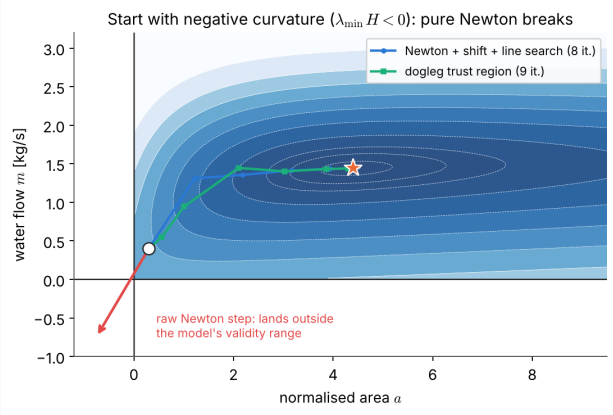


Same start $(0.5, 0.5)$:

method	iterations
GD, fixed $\alpha = 0.01$	> 500
GD, line search	242
Newton (safeguarded)	7
Dogleg	9

- The curve for Newton is almost **vertical**: $10^{-1} \to 10^{-3} \to 10^{-6}$.
- But each iteration needs a Hessian (here by finite differences: 206 function calls in total).

When Newton fails: negative curvature



- At small area, f is **not convex**: H has eigenvalues -28 and $+154$.
- The "parabola" opens downward in one direction: its stationary point is a **saddle**, not a minimum.
- Pure Newton jumps to $a < 0$, $m < 0$: physically meaningless (the model is invalid there).

Two cures
(1) Fix H and line-search, (2) **limit the step** to a region where the model is trusted.

Trust region: how far do we believe the model?

- Model $m(p) = f + g^\top p + \frac{1}{2} p^\top B p$ is trusted only for $\|p\| \le \Delta$ (trust-region radius).
- After the step, compare what the model **promised** with what actually happened:

$$\rho = \frac{f(x) - f(x+p)}{m(0) - m(p)} = \frac{\text{actual decrease}}{\text{predicted decrease}}$$

ρ	meaning	action
$\rho < 0.25$	model was poor	**shrink** Δ ($\times \frac{1}{4}$)
$\rho > 0.75$ and step on the boundary	model was good	**expand** Δ ($\times 2$)
$\rho > 0.1$	real improvement	**accept** the step
otherwise		reject, try a smaller region

The remaining question: which step p inside the region? Solving the subproblem exactly is expensive; the **dogleg** gives a cheap, good answer.

The dogleg path

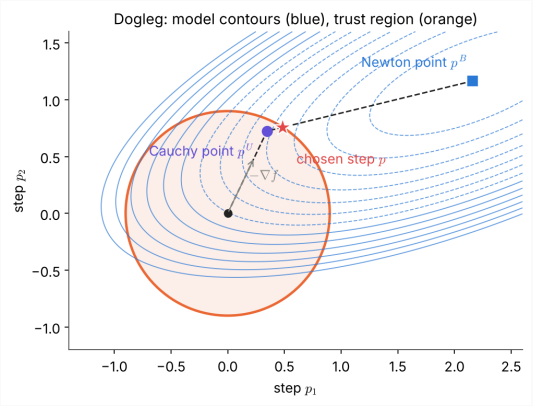
Two candidate points in step space:

- **Cauchy point** p^U: best step along $-\nabla f$ (safe, slow).
- **Newton point** $p^B = -B^{-1}g$ (fast, only trustworthy nearby).

The **dogleg path** goes $0 \to p^U \to p^B$ (a bent line, like a dog's hind leg). The step is where the path leaves the trust region.

- $\|p^B\| \le \Delta$: take the Newton step.
- $\|p^U\| \ge \Delta$: truncate steepest descent.
- otherwise: stop on the sphere along $p^U \to p^B$.

Small region ⇒ gradient-like; large region ⇒ Newton.



Dogleg: the code

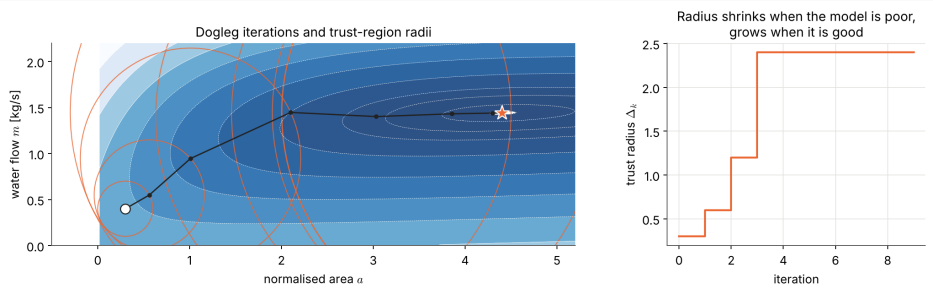
```
def dogleg_step(g, B, Delta):
    """Step inside the trust region |p| <= Delta along the 'dogleg' path Cauchy -> Newton."""
    p_newton = -np.linalg.solve(B, g)
    if np.linalg.norm(p_newton) <= Delta:                    # Newton step fits: take it
        return p_newton
    p_cauchy = -(g @ g) / (g @ B @ g) * g                    # minimiser along steepest descent
    if np.linalg.norm(p_cauchy) >= Delta:                    # even that is too long: truncate
        return -Delta * g / np.linalg.norm(g)
    d = p_newton - p_cauchy                                  # walk from Cauchy towards Newton ...
    a, b, c = d @ d, 2 * p_cauchy @ d, p_cauchy @ p_cauchy - Delta ** 2
    tau = (-b + np.sqrt(b * b - 4 * a * c)) / (2 * a)        # ... until we hit |p| = Delta
    return p_cauchy + tau * d
```

The step: Newton if it fits, truncated Cauchy if even that is too long, otherwise the intersection of the dogleg path with $\|p\| = \Delta$ (a quadratic equation for τ).

Dogleg: the trust-region loop

```python
def trust_region_dogleg(f, x0, Delta0=0.3, Delta_max=3.0, n_iter=60, tol=1e-6):
    x, Delta = np.array(x0, float), Delta0
    path, radii = [x.copy()], [Delta]
    for k in range(n_iter):
        g, H = numgrad(f, x), numhess(f, x)
        if np.linalg.norm(g) < tol:
            break
        lam_min = np.linalg.eigvalsh(H)[0]
        B = H + max(0.0, 1e-6 - lam_min) * np.eye(len(x))  # make the model convex
        p = dogleg_step(g, B, Delta)
        predicted = -(g @ p + 0.5 * p @ B @ p)             # decrease promised by the model
        actual = f(x) - f(x + p)                           # decrease really obtained
        rho = actual / predicted
        if rho < 0.25:
            Delta *= 0.25                                  # model was poor: shrink the region
        elif rho > 0.75 and np.linalg.norm(p) > 0.99 * Delta:
            Delta = min(2 * Delta, Delta_max)              # model was good: expand it
        if rho > 0.1:                                      # accept the step only if it helped
            x = x + p
        path.append(x.copy())
        radii.append(Delta)
    return x, np.array(path), np.array(radii)
```

Dogleg on P3 from the non-convex start



Start $(0.3, 0.4)$ where pure Newton breaks: radius $0.3 \to 0.6 \to 1.2 \to 2.4$ (the model is good), converging in 9 iterations. When the model is poor, the radius would shrink instead.

Newton-type methods in `scipy`

```
from scipy.optimize import minimize
g = lambda x: numgrad(cost_exchanger, x);  H = lambda x: numhess(cost_exchanger, x)
res = minimize(cost_exchanger, x0, jac=g, hess=H, method="trust-exact")   # 7 its.
```

P3 from (0.5, 0.5)	`scipy` method	iterations	f evals
trust region, exact subproblem	`trust-exact`	7	8
trust region, dogleg	`dogleg`	stops at 0	1
Newton with CG	`Newton-CG`	8	11
quasi-Newton (BFGS, H estimated)	`BFGS`	11	36

- `scipy`'s `dogleg` **requires** $H > 0$: here it stops at once (non-PSD Hessian). Our version shifts H; `trust-exact` handles negative curvature natively.
- **BFGS** builds H from gradients only: the standard choice when Hessians are unavailable.

Nelder–Mead simplex: no derivatives, no probabilities

- $n+1$ points form a **simplex** (triangle in 2-D).
- Sort by f; the **worst** vertex is replaced by a better point:

1. **reflect** through the centroid;
2. if great: **expand**;
3. if bad: **contract**;
4. if still bad: **shrink** towards the best.

(Unrelated to the *LP simplex* of linear programming.)



Nelder–Mead: the code (1/2)

```
def nelder_mead(f, x0, size=0.5, n_iter=100, tol=1e-8):
    """Nelder-Mead simplex: n+1 points, no derivatives."""
    n = len(x0)
    S = np.array([x0] + [x0 + size * np.eye(n)[i] for i in range(n)], float)
    F = np.array([f(s) for s in S])
    history = [S.copy()]
    for _ in range(n_iter):
        order = np.argsort(F)
        S, F = S[order], F[order]                          # best first, worst last
        if abs(F[-1] - F[0]) < tol:
            break
        c = S[:-1].mean(axis=0)                            # centroid of all but the worst
        xr = c + (c - S[-1])                               # reflection
        fr = f(xr)
        if fr < F[0]:                                      # very good: try to go further
            xe = c + 2 * (c - S[-1])                       # expansion
            fe = f(xe)
            S[-1], F[-1] = (xe, fe) if fe < fr else (xr, fr)
        elif fr < F[-2]:                                   # acceptable: keep the reflection
            S[-1], F[-1] = xr, fr
```

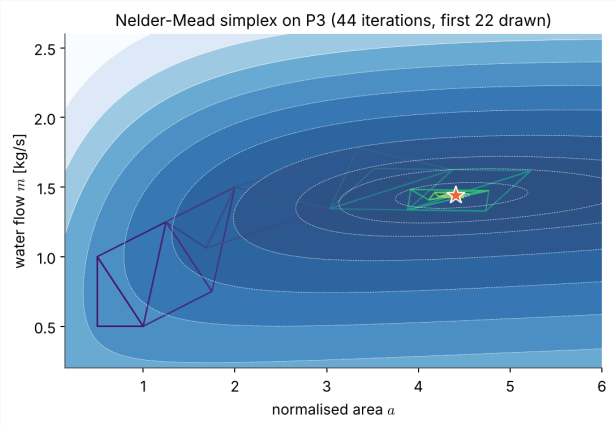
Reflection, expansion and the “keep the reflection” case.

Nelder–Mead: the code (2/2)

```
        else:                                                # bad: contract towards the centroid
            xc = c + 0.5 * (S[-1] - c)
            fc = f(xc)
            if fc < F[-1]:
                S[-1], F[-1] = xc, fc
            else:                                            # shrink everything towards the best
                S[1:] = S[0] + 0.5 * (S[1:] - S[0])
                F[1:] = [f(s) for s in S[1:]]
        history.append(S.copy())
    best = np.argmin(F)
    return S[best], F[best], history
```

Contraction, shrink, and the bookkeeping of the simplex history (used for the figure).

Nelder–Mead on P3

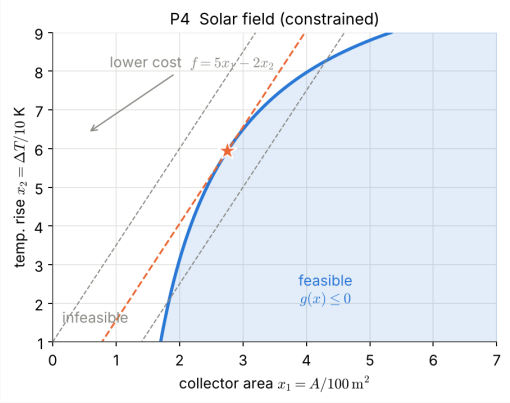


- The triangle **crawls, stretches and rotates** down the valley. Colour = iteration.
- Manual: 47 iterations, 89 evaluations. `scipy Nelder-Mead`: 63 iterations, 120 evaluations. Both reach $f = -93.575$.
- Compare: Newton 7 iterations, but each needs derivatives.

When to use it

Few variables, noisy or non-smooth f (experiments, simulations), no derivatives. Degrades beyond ~ 10 variables; can stall on rough problems.

Constrained problem P4: where is the optimum?



- Lower cost is up-left (less area, hotter heat), but the demand constraint blocks it: the optimum sits **on the boundary** $g(x) = 0$.
- There ∇f and ∇g are **parallel**:

$$\nabla f(x^*) + \mu\, \nabla g(x^*) = 0, \qquad \mu \ge 0$$

- $\mu^* = 5$ **k$/yr** per unit of g (100 m^2): the **shadow price** of the constraint, i.e. what one more unit of required area costs.

Idea shared by the next methods
Turn the constrained problem into easier ones: **linear programs** (cutting plane) or **unconstrained** problems (penalty, barrier).

Cutting plane (Kelley, 1960): replace g by its tangents

1. Solve the **LP**: minimise $c^\top x$ in the box, subject to all cuts so far (none at first).
2. If its solution x_k satisfies $g(x_k) \le \text{tol}$: **stop**.
3. Otherwise add the **cut** (tangent plane of g at x_k):

$$g(x_k) + \nabla g(x_k)^\top (x - x_k) \le 0$$

and repeat.

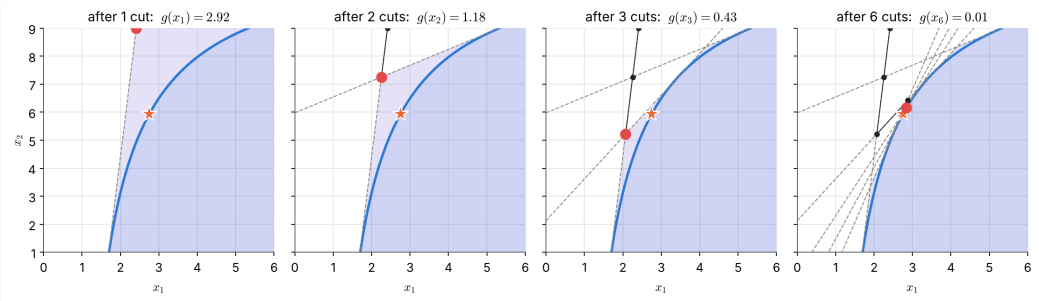
Why it is safe

If g is **convex**, its tangent lies below it: $g(x) \ge g(x_k) + \nabla g(x_k)^\top (x - x_k)$. Every feasible point satisfies the cut, so cuts remove **only infeasible** points.

Here f is linear, so only g is linearised. For a nonlinear convex f: minimise a new variable $t \ge f(x)$ (the *epigraph trick*) and cut f as well.

Cost: one small LP per iteration (`scipy.optimize.linprog`).

Cutting plane on P4: the polygon closes in on the boundary



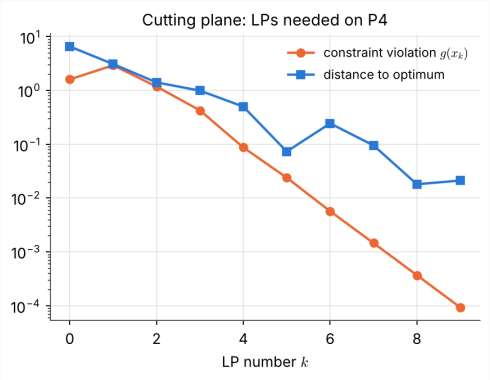
Violet: region allowed by the cuts so far. Red dot: the LP solution x_k. Each cut slices off the previous (infeasible) vertex; iterates approach the optimum $\star$ from **outside**.

Cutting plane: the code

```
def cutting_plane(c, g, x0, lo, hi, tol=1e-4, max_iter=50):
    """Kelley's cutting plane for  min c.x  s.t. g(x) <= 0  (g convex), lo <= x <= hi."""
    A_cuts, b_cuts = [], []
    x = np.array(x0, float)
    path = [x.copy()]
    for k in range(max_iter):
        gx = g(x)
        if np.max(gx) <= tol and k > 0:                    # LP solution is feasible: done
            break
        J = numjac(g, x)
        for i in range(len(gx)):                           # tangent plane (cut) of each g_i
            A_cuts.append(J[i])
            b_cuts.append(J[i] @ x - gx[i])                # g_i(x_k) + J_i (x - x_k) <= 0
        res = linprog(c, A_ub=np.array(A_cuts), b_ub=np.array(b_cuts),
                      bounds=list(zip(lo, hi)), method="highs")
        x = res.x                                          # new LP vertex
        path.append(x.copy())
    return x, np.array(path), (np.array(A_cuts), np.array(b_cuts))
```

The Jacobian of g is obtained like `numgrad`, one row per constraint (`numjac`). The LP solver is HiGHS inside `linprog`.

Cutting plane: result and limits

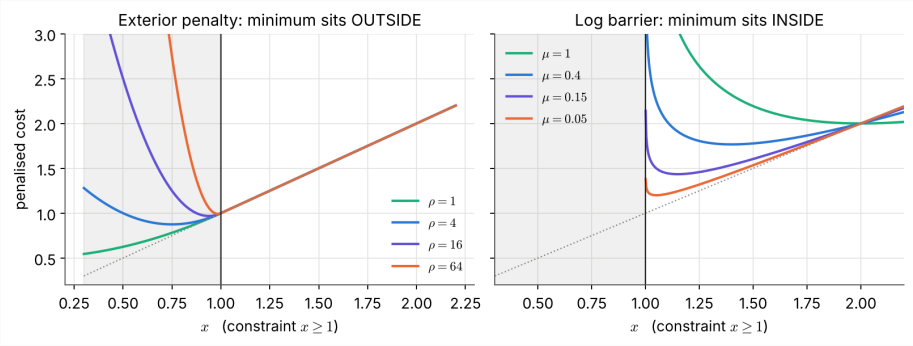


- 9 LPs reach $x = (2.744, 5.917)$, i.e. $A = 274$ m^2, $\Delta T = 59.2$ K, violation $< 10^{-4}$.
- Violation falls steadily; the distance to the optimum is erratic (iterates jump from vertex to vertex).
- The cuts accumulate: LPs grow every iteration.

Limits

Needs **convexity** (otherwise a cut can remove the true optimum). Slow on curved boundaries. Iterates are infeasible until the end.

Penalty and barrier: forget the constraint, change the objective



Toy case: minimise x subject to $x \geq 1$.

Exterior penalty

$Q(x;\rho) = f(x) + \dfrac{\rho}{2}\max\{0, g(x)\}^2$

cost for **violating**: $\rho \uparrow \infty$.

Log barrier

$B(x;\mu) = f(x) - \mu \log\big(-g(x)\big)$

wall at the boundary: $\mu \downarrow 0$.

Exterior versus interior

	Exterior penalty	**Log barrier**
Iterates	**infeasible** (outside), approach from outside	**feasible** (inside), approach from inside
Starting point	anything	must satisfy $g < 0$
Stopping early	slightly violates the constraint	a usable (feasible) design
Multiplier estimate	$\hat{\mu} = \rho \max\{0, g(x_\rho)\}$	$\hat{\mu} = -\mu / g(x_\mu)$
Equalities	natural	not directly
Price	ill-conditioned Hessian, $\sim \rho$	ill-conditioned Hessian, $\sim 1/\mu$

Procedure: solve min Q (or min B) for a **sequence** $\rho = 10, 10^2, \dots$ (or $\mu = 1, 0.3, \dots$), each time starting from the previous solution (*warm start*), with any unconstrained solver (Newton, BFGS, ...).

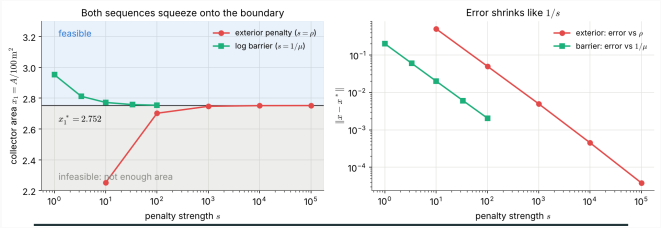
Penalty and barrier: the code

```python
def penalised(f, g, rho=None, mu=None):
    """Return an UNCONSTRAINED function: exterior penalty (rho) or log-barrier (mu)."""
    if rho is not None:
        return lambda x: f(x) + 0.5 * rho * np.sum(np.maximum(0.0, g(x)) ** 2)
    def barrier(x):
        gx = g(x)
        return f(x) - mu * np.sum(np.log(-gx)) if np.all(gx < 0) else np.inf
    return barrier


def solve_sequence(f, g, x0, params, kind="exterior"):
    """Solve a sequence of unconstrained problems; warm-start each from the previous one."""
    x, out = np.array(x0, float), []
    for p in params:
        F = penalised(f, g, rho=p) if kind == "exterior" else penalised(f, g, mu=p)
        x, _ = newton_inner(F, x)
        out.append(x.copy())
    return np.array(out)
```

`newton_inner` is the damped Newton of the previous section (returning $+\infty$ outside the barrier's domain is how it keeps the iterates feasible).

Penalty and barrier on P4



exterior: ρ	10	10^2	10^3	10^4	10^5
A [m^2]	225	270	275	275	275
$\hat{\mu} = \rho\, g$	5.000	5.000	5.000	5.000	5.000
barrier: $1/\mu$	1	3.3	10	33	100
A [m^2]	295	281	277	276	275
$\hat{\mu} = -\mu/g$	5.000	5.000	5.000	5.000	5.000

Both estimates recover the shadow price $\mu^* = 5$ k\$/yr. (Here f and g are linear in x_1, so x_2 is exact from the first subproblem.)

The price: conditioning, and a warning about scaling

- As $\rho \to \infty$ the penalty term $\frac{\rho}{2}\,\nabla g\,\nabla g^{\top}$ dominates the Hessian of Q: one huge eigenvalue, the rest small. The **condition number grows like** ρ (like $1/\mu$ for the barrier) when fewer constraints are active than variables.
- Consequences: Newton's linear systems become inaccurate, BFGS slows, steps get tiny near the boundary (we stopped the barrier at $\mu = 0.01$ in finite-difference arithmetic).
- **Scale your variables** ($x_1 = A/100$ m^2, $x_2 = \Delta T/10$ K): with variables of very different size the penalty acts unevenly.

How practitioners avoid it

Exact penalties (ℓ_1), the **augmented Lagrangian**, and **sequential quadratic programming** (SQP) use the multiplier estimates to reach the solution with a *finite* penalty.

Constrained problems in `scipy`

```python
from scipy.optimize import minimize
cons = [{"type": "ineq", "fun": lambda x: -g_solar(x)}]            # scipy wants  c(x) >= 0
res  = minimize(cost_solar, [3, 3], constraints=cons, bounds=[(0, 6), (1, 9)], method="SLSQP")
```

Method	kind	A [m^2]	ΔT [K]	work
exact solution	–	275.2	59.37	–
cutting plane (manual)	LP sequence	274.4	59.17	9 LPs
exterior penalty, $\rho = 10^5$	unconstrained seq.	275.2	59.37	5 Newton solves
log barrier, $\mu = 0.01$	unconstrained seq.	275.4	59.37	5 Newton solves
`SLSQP`	SQP	275.2	59.37	12 iterations
`trust-constr`	interior point	275.2	59.37	24 iterations

`SLSQP` is a sequential quadratic programming method: the professional descendant of the ideas on this section.

The methods side by side

Method	Needs	Strength	Weakness	Thermal us
Golden section	f (1-D)	robust, guaranteed if unimodal	one variable; one valley	insulation t length
Metropolis / SA	f	crosses barriers; any f	many evaluations; tuning of T, step	placement,
Great Deluge	f, a target	very simple; goal-oriented	fails if target unreachable	layout with goal
Gradient descent	$f, \nabla f$	simple; low memory	slow in narrow valleys; step size	first runs, h
Newton	$f, \nabla f, H$	quadratic convergence	breaks if H not PD; costly H	smooth de n
Dogleg (trust region)	$f, \nabla f, H$	robust Newton	needs H; code is longer	calibration, conditioned
Nelder–Mead	f	no derivatives; compact	scales badly beyond $\sim$ 10 variables	experiment simulations
Cutting plane	$f, g, \nabla g$	only LPs; certificates	convex only; slow	convex de constraints
Penalty / barrier	any solver + g	reuses unconstrained code	ill-conditioning; scaling	quick const ies

Cost on P3: manual codes versus `scipy`

manual (this seminar)			`scipy.optimize`		
method	its.	f calls	method	its.	f calls
GD + line search	242	3102	`Nelder-Mead`	63	120
Newton	7	206	`BFGS`	11	36
Dogleg	9	218	`Newton-CG`	8	11*
Nelder–Mead	47	89	`trust-exact`	7	8*

* Derivatives supplied (by our finite-difference code) are not counted here.

- Every method reaches $f^* = -93.575$ k\$/yr. The differences are the **cost of getting there**.
- Our manual counts are inflated by finite differences: a gradient costs $2n$ and a Hessian $4n^2$-ish calls. Analytic or automatic derivatives would shrink them by orders of magnitude.

Choosing a method: a short guide

Ask about the function

- **One variable, unimodal?** Golden section.
- **Several valleys?** Annealing / Deluge / multi-start, then a local polish.
- **Noisy or non-differentiable?** Simplex (or annealing).
- **Smooth, derivatives cheap?** Newton-type; trust region if the Hessian can be indefinite; BFGS if you only have gradients.

Ask about the constraints

- **None:** the methods on the left.
- **Bounds only:** L-BFGS-B, bounded simplex.
- **Convex, few variables:** cutting plane; or a convex solver.
- **General nonlinear:** SQP (`SLSQP`) or interior point (`trust-constr`).
- **Quick study:** penalty / barrier on top of any unconstrained solver.

Always: **plot first**, **scale** the variables, **check** the result against physics, and use **several starting points**.

Five messages to take home

1. **Look at f first.** Count valleys, check smoothness, note scales. The landscape chooses the method.
2. **Information buys speed.** Values only (robust, slow) $\rightarrow$ gradient $\rightarrow$ Hessian (fast, fragile). Safeguards (line search, trust region) make the fast methods reliable.
3. **Escaping local minima costs evaluations.** Stochastic methods need tuned knobs (T, step, target) and **many seeds**; keep the best point seen.
4. **Constraints are handled by reduction**: to LPs (cutting plane), to sequences of unconstrained problems (penalty, barrier) or to QPs (SQP). Read the **multipliers**: they are shadow prices.
5. **Code them once, compare with `scipy`.** A 20-line manual version explains the method; the library version is what you deploy.

Hands-on exercises

1. **P1:** double the energy price. How does the optimal thickness move? Use golden section and check with the plot.
2. **P2:** add a fifth cooled clamp. How many local minima appear? Compare fixed-T Metropolis and SA at $T = 1$, 5, 20 over 30 seeds.
3. **P3:** use A in m^2 instead of $a = A/50$. What happens to the condition number and to gradient descent? Why does Newton not care?
4. **P3:** start Newton from 20 random points. Where does it fail? Does the dogleg fail there too?
5. **P4:** raise the demand to 150 kW. Predict how the objective changes using $\mu^* = 5$; verify with SLSQP.
6. Replace `numgrad` by an analytic gradient for P1 and count the function calls saved.

References

- J. Nocedal and S. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006 (line search, Newton, trust region and dogleg: ch. 3–4; penalty and barrier: ch. 17).
- A. Bejan, G. Tsatsaronis and M. Moran, *Thermal Design and Optimization*, Wiley, 1996.
- J. Kiefer, "Sequential minimax search for a maximum", *Proc. AMS* 4, 502–506, 1953 (golden section).
- N. Metropolis et al., "Equation of state calculations by fast computing machines", *J. Chem. Phys.* 21, 1087–1092, 1953.
- S. Kirkpatrick, C. Gelatt and M. Vecchi, "Optimization by simulated annealing", *Science* 220, 671–680, 1983.
- G. Dueck, "New optimization heuristics: the great deluge algorithm and the record-to-record travel", *J. Comput. Phys.* 104, 86–92, 1993.
- J. Nelder and R. Mead, "A simplex method for function minimization", *Computer J.* 7, 308–313, 1965.
- J. Kelley, "The cutting-plane method for solving convex programs", *J. SIAM* 8, 703–712, 1960.
- A. Fiacco and G. McCormick, *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*, Wiley, 1968.
- Code of this seminar: `problems.py`, `methods.py`, figure scripts (all objectives `f(x)` with `x` a vector).

Thank you

Questions and discussion